Radioactive Decay

Son Thanh Truong
Student ID: 100972716

Carleton University
PHYS 2007
Lab Section: C
Lab Partner: Lauren Gilchrist
Date of Report Submission: November 7, 2026

November 7, 2026

1 Introduction

This experiment has 4 parts. Background Radiation, The Half-Life of Barium-137m, Pb Absorption Coefficient, and The Inverse-Square Law. In these experiments we aim to learn about the properties of nuclear decay, and familiarize with Geiger-Muller counters. We will measure the background radiation in the lab room compare it with A poisson distribution. We will determine the half-life of Barium-137m, an unstable isotope. We will find the absorbtion coefficient of lead and finally we will investigate the Inverse-Square Law.

1.1 Background Radiation

Background radiation is all around us, even in our physics labs. And since we will be measuring radiation to model and fit plots, in order to do that accurately we need to measure the background radiation so then we could use python to subtract it from our measurement for the best fit possible. Also the data collected will model a Poisson distribution. The sampling was 5 seconds per sample for 100 samples totaling to 500 seconds for the measurement.

1.2 The Half-Life of Barium-137m

The goal of this part is to measure the half-life of Barium-137m.The measurement part is almost identical to the background radiation part. The rate of measurement is 5 seconds per sample for 240 samples totaling at 1200 seconds. In the analysis using code, all of the samples were subtracted/corrected by the average of the background radiation. Since the total size of the number of samples aren't the same. But more on that later.

1.3 Pb Absorption Coefficient

The goal of this part was to Find the absorbtion coefficient of lead. According to this equation:

$$I = I_o * np.exp(-u * x) \tag{1}$$

Where I is the outputed intensity and I_o is the initial intensity, u is the absorption coefficient, and x is the thickness of the lead.

1.4 The Inverse-Square Law

Here is the equation for the Inverse-Square Law that was given:

$$I = \frac{a}{(x-b)^2} + c \tag{2}$$

Where I is the intensity at distance x from the source, and a, b, and c are parameters to be determined. After trying out the fit for this equation and faiing even after setting b and c to zero, it was decided that b and c should be set to zero manually by using a simpler equation:

$$I = \frac{a}{x^2} \tag{3}$$

This was what was used to find the fit. In the end, it only worked with this equation.

2 Experimental Setup

The equiptments used in this lab includes:

- Geiger-Muller counter
- Radioactive source (Cs137, Co60)
- Cs-137/Ba-137m Isotope Generator
- Lead Plates
- Vernier Caliper or Micrometer
- Retort Stand
- Tongs, Petrie Dish, Paper Filter
- Lab Quest Mini Interface
- Logger Pro Software

The GM counter will be used on all of the 4 parts of the lab as a tool needed to measure the radiation for all of our data.

3 Observations

3.1 Background Radiation

For this part, the GM counter was placed stationary with no major radiactive sources nearby. Then 100 samples of data was recorded with a duratoin of 5 seconds per sample. Below is a sample for the data for this part of the lab:

Time (s)	Latest: Radiation (Counts)
5	2
10	0
15	5
20	0
25	6
30	3
35	4
40	3
45	1
50	3

3.2 The Half-life of Barium-137m

One round of continuous measurements of 5s/sample for 1200s in total. This is only a sample of the measurement.

Time (s)	Latest: Radiation (Counts)
5	245
10	218
15	251
20	247
25	234
30	235
35	205
40	220
45	221
50	239

3.3 Pb Absorption Coefficient

There are 5 different square lead sheets of different thinknesses. Sheets were stacked from zero sheets to five. Each time one sample of 200s duration was recorded. Here is the results of all of the data gathered. The most important colume that is being graphed is The Thickness, and the Counts.

Thickness (cm)	Counts	Average Sheet Thickness (mm)	Trial 1	Trial 2	Trial 3	Minus Z
0.000000	397	0.000000	NaN	NaN	NaN	
1.770000	361	1.770000	1.920000	1.890000	1.980000	
5.870000	386	4.100000	3.770000	4.200000	4.810000	
7.523333	360	1.653333	1.840000	1.810000	1.820000	
10.136667	323	2.613333	2.780000	2.740000	2.770000	
11.813333	270	1.676667	1.840000	1.840000	1.830000	

4 Data Analysis

4.1 Background Radiation

For the Background Radiation, we took 100 samples of 5s duration. The result for the average/mean of backround radiation rate using python's numpy library, np.mean is: 2.68 +/- 1.54 Counts per 5 seconds. Using the python code the sample variance was given as 2.38 $(Countsper5seconds)^2$. The standard deviation is the square root of that, 1.54. Here is a graph of the background radiation total counts in 5s versus how many times that happened inthe Y-axis.

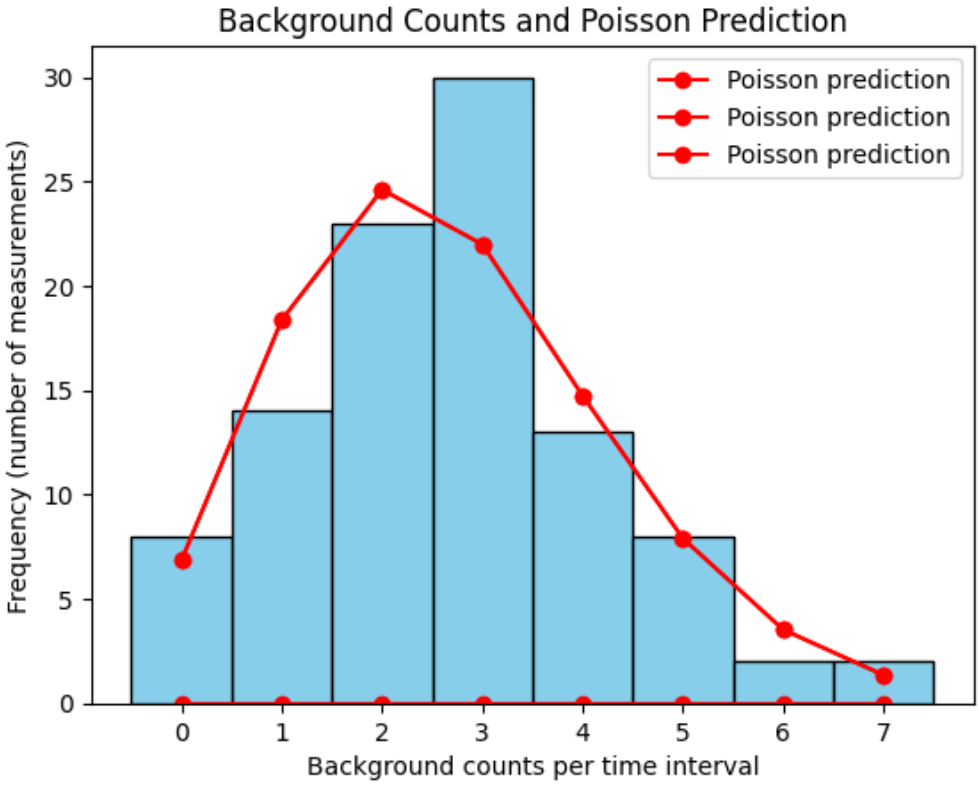


Figure 1: Radioactive decay of Barium-137m over time.

4.2 The Half-Life of Barium-137m

To find the half-life of Barium-137m, the Barium radioactive material was put very close to The GM counter. The sample rate was 5s/sample. When the data was analyzed, the background radiation counts data recorded in the previous section were subtracted easily using programming. Basically, the mean of the Background Radiation were subtracted from each sample. This is very easily done using Python. Here is the line of code used to subtract the Barium Radiation recording:

```
# 5. Subtract the corrected average background
corrected_counts = HL_corrected - bkgAvg_corrected
```

There were other calibrations to correct the data for a better fit, but the resulting change was so small that I believe that it could be neglected but still make a great fit. The Calibration is called Geiger Counter Dead Time. Here is the equation for that:

$$N_{true} = \frac{n_{observed}}{1 - n_{observed}\tau} \tag{4}$$

N_{true} is the true count, $n_{observed}$ is the observed count, and τ is the dead time of the Geiger counter.

After subtracting the background radiation, The data was plotted and here it is: The

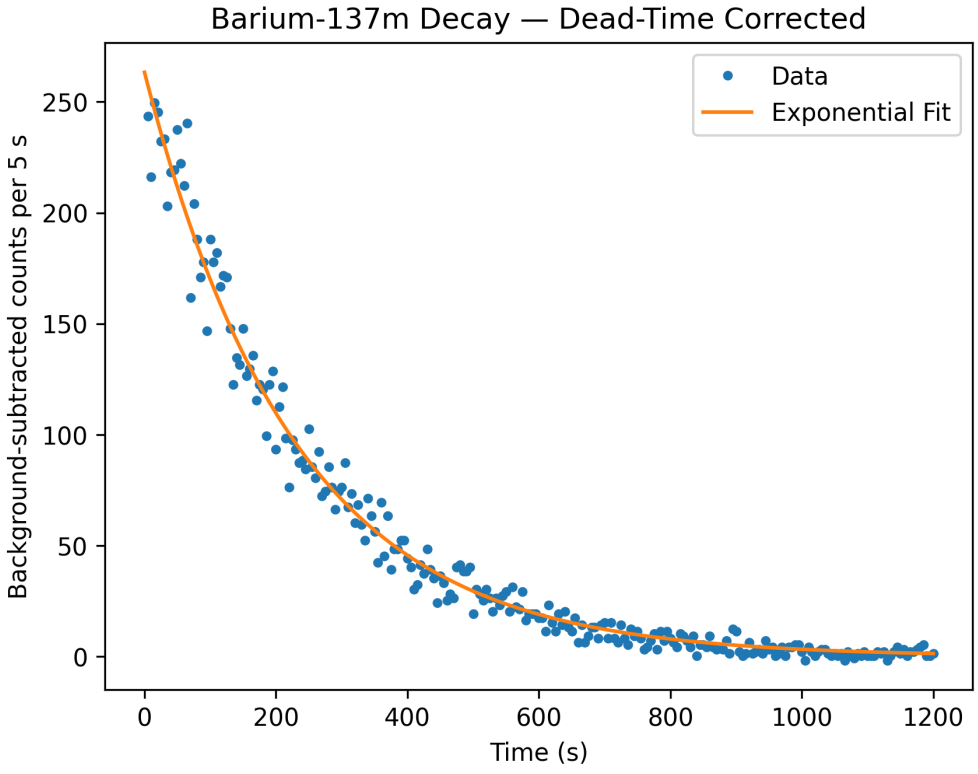


Figure 2: Radioactive decay of Barium-137m over time.

half-life of Barium-137m is: Half-Life = ln2/lambda = ln2/0.0043773832515909585 is ap-

proximately 158.35 seconds For the error calculation:

$$\sigma_{T_{1/2}} = |\frac{dT}{d\lambda}|\sigma_\lambda = |\frac{d(ln2/\lambda)}{d\lambda}|\sigma_\lambda = |-ln2/\lambda^2|\sigma_\lambda \tag{5}$$

where $\sigma_{T_{1/2}}$ is the error in the half-life and σ_λ is the error in the decay constant. When the Lambda is 0.0043773832515909585 and the error on lambda is 5.76137806e-05,we get:

$$\sigma_{T_{1/2}} = |-ln2/0.0043773832515909585^2|5.76137806e-05 = 2.04812 \text{ s} \tag{6}$$

The final answer is: $T_{1/2} = 158.35 \pm 2.05$ s

4.3 Pb Absorption Coefficient

This part is composed of changing the thickness of the lead that the radiation has to go through and measuring what that effect has on the intensity of the radiation. The lead involved composes of different square sheets with different thicknesses but average about 1mm in thickness in zip lock bags. The data was them collected, graphed and fitted. Below is the fitted graph showing the data points and the fitted curve. The result is that the absorption coefficent is 0.02 +- 0.005 cm^{-1}. The error on the coefficient was exported through the variable 'pcov' from the curve fit.

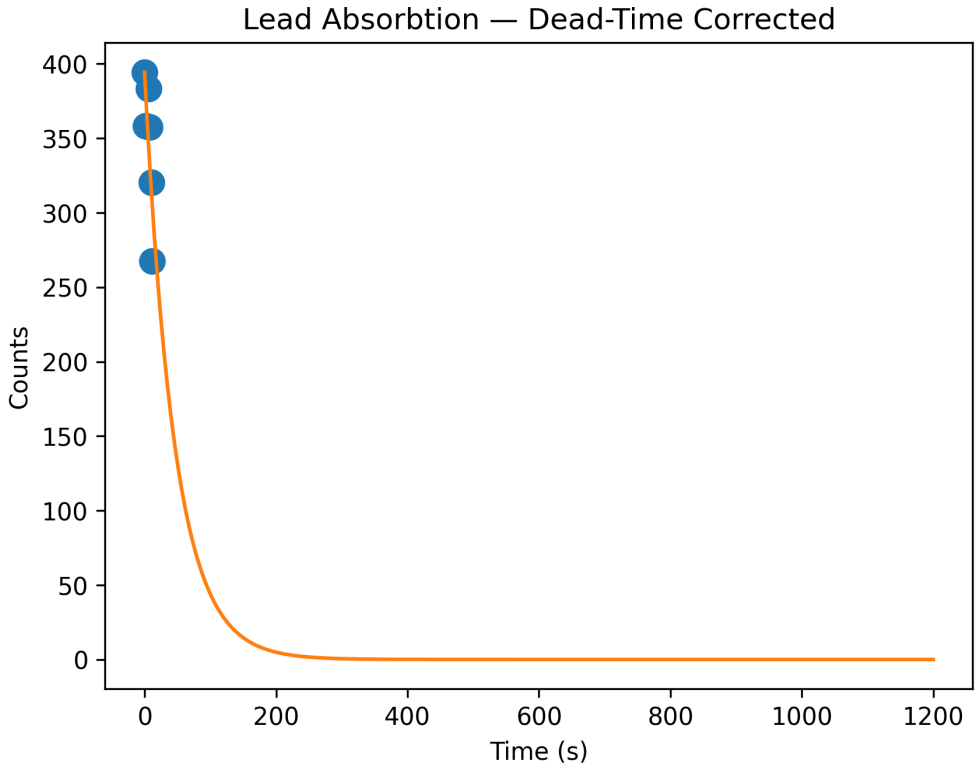


Figure 3: Pb absorption coefficient measurement.

4.4 Inverse-Square Law

The analyis for the Inverse-Square Law is very similar to previous analysis. After data is collected during the experiment, the data was sampled into a one dimensional array. Two arrays of the same size were created for graphing. The Background mean was used for correction. At that point the Curve Fit function was used. The initial function from the lab manual that I tried was:

```
def ISL(x,a,b,c,n):
    return a/(x-b)**n + c
```

After calling Curve Fit many times with no luck even after setting b, and c to zero in the call. I decided to set them to zero manually by removing them out of the equation. The new function I used was:

```
def ISL2(x,a,n):
    return a/x**n
```

Then I called:

```
popt, pcov = curve_fit(I, thickness, Counts, [1])
```

Where the very last parameter [1] is my guess for N. I ended up getting N = 1.53. The error on N is 0.0558. Thus the total result is $N = 1.53 +/- 0.0558$.

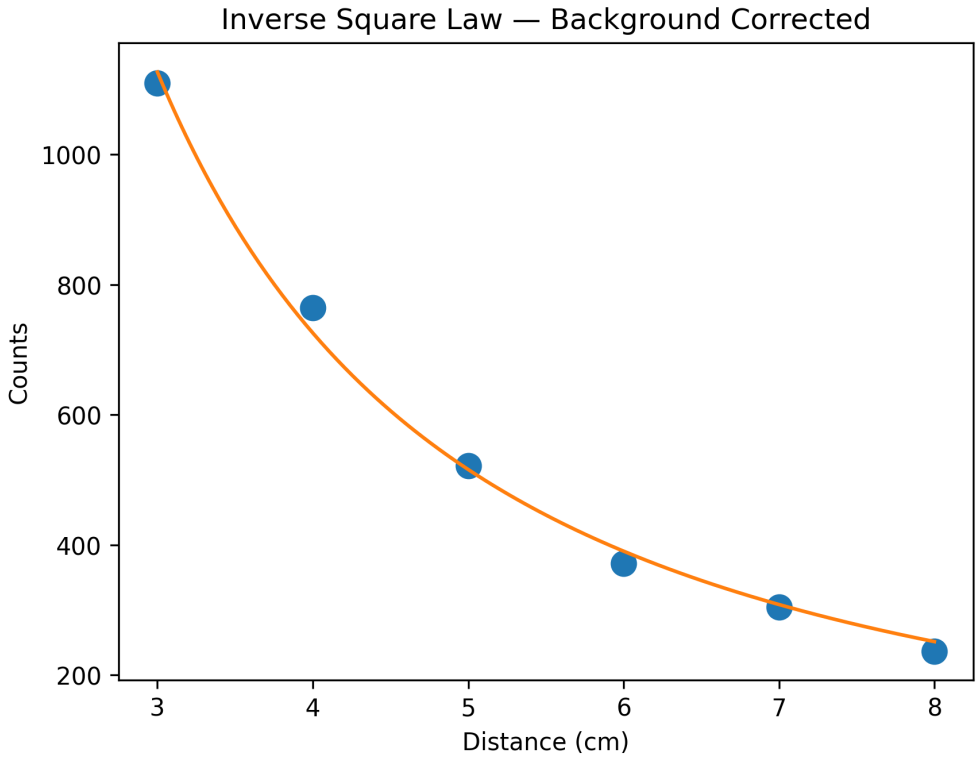


Figure 4: Inverse-square law measurement.

5 Discussion

The results of this lab are as follows: with the background radiation part of the lab, it was graphed the data using bins and it seems like it does look like a poisson distribution when comparing. The mean for this data set is 2.68 +/- 1.54 Counts per 5 seconds.

With the half-life calculation, the final value, the half-life of this process is: 158.35 +/- 2.05 seconds. That's reasonably good, considering the accepted value is 153.12 seconds.

Part 3 the Lead Absorption Coefficient was: $0.02 +/- 0.005 mm^{-1} = 0.2 +/- 0.05 cm^{-1}$ The expected value should be $1.26 cm^{-1}$. So something went wrong there, I wonder if we switched cm for mm. Because everything is measured in mm, but the final value is cm. This is a lesson for me for next time for sure.

And the final part, the Inverse-Square Law's N constant is: $1.53 +/- 0.0558$ N should be 2, but it's calculated to be 1.53. the biggest source of error I see here is that I actually used a 20 cm ruler to match distances with the one meter ruler which is very imprecise. I was in a hurry to get it done before time ran out. I should've put on my mask, stay more calm to make way better measurements.

Overall, I felt very successful and optimistic with this lab because the results seem to be very accurate. because all the errors are very low. I wonder if's it because I was the only one taking measurements for some of the parts. I'm slower than my lab partner but I'm actually very careful and I do things in order and with precision. When my lab partner came I had already finished taking data. Data for our other labs were never this precise, there were multiple problems with the data and the fits doesn't look good. I guess it pays to be slow and careful.

In the next lab, I will try to understand the theory beforehand so I wouldn't be so confused meaning I could resolve my partner's question and not have to rely on our busy TAs. Also on another note, the use of masks seems to lower my anxiety by quite a lot. I was able understand our professor better when he lectures, be more focused during the experiment and have way more mental stamina. Masks are the way to go.

6 Excel/Jupyter Files

A Experimental Data

B Background Radiation Data

Time (s)	Latest: Radi
5	2
10	0
15	5
20	0
25	6
30	3
35	4
40	3
45	1
50	3
55	1
60	6
65	2
70	1
75	3
80	2
85	2
90	4
95	1
100	3
105	3
110	5
115	0
120	3
125	0
130	1
135	2
140	1
145	5
150	3
155	7
160	4
165	3
170	4
175	1
180	5
185	2
190	3
195	1
200	0
205	1
210	4
215	4
220	3
225	3
230	1
235	3

240	2
245	3
250	4
255	3
260	1
265	2
270	2
275	2
280	3
285	4
290	1
295	2
300	4
305	7
310	1
315	4
320	2
325	2
330	3
335	2
340	0
345	0
350	3
355	3
360	2
365	2
370	5
375	3
380	1
385	2
390	5
395	5
400	3
405	3
410	2
415	2
420	4
425	3
430	4
435	3
440	5
445	3
450	2
455	3
460	0
465	2
470	3
475	3

480	2
485	3
490	2
495	3
500	4

C Barium-137m Half-Life Data

Time (s)	Latest: Radi
5	245
10	218
15	251
20	247
25	234
30	235
35	205
40	220
45	221
50	239
55	224
60	214
65	242
70	164
75	206
80	190
85	173
90	180
95	149
100	190
105	180
110	184
115	169
120	174
125	173
130	150
135	125
140	137
145	134
150	150
155	129
160	132
165	138
170	118
175	125
180	123
185	102
190	125
195	131
200	96
205	115
210	124
215	101
220	79
225	100
230	96
235	90

240	91
245	87
250	105
255	88
260	83
265	95
270	75
275	77
280	88
285	79
290	69
295	77
300	79
305	90
310	70
315	76
320	63
325	71
330	62
335	55
340	74
345	66
350	59
355	45
360	72
365	48
370	66
375	42
380	51
385	51
390	55
395	55
400	47
405	43
410	33
415	35
420	44
425	40
430	51
435	42
440	38
445	27
450	39
455	36
460	28
465	31
470	29
475	43

480	44
485	41
490	41
495	43
500	22
505	33
510	31
515	28
520	33
525	29
530	23
535	29
540	26
545	30
550	32
555	23
560	34
565	25
570	24
575	32
580	19
585	22
590	22
595	22
600	20
605	20
610	14
615	26
620	18
625	14
630	22
635	17
640	23
645	16
650	14
655	20
660	9
665	17
670	9
675	12
680	16
685	16
690	11
695	17
700	18
705	11
710	18
715	11

720	9
725	17
730	11
735	8
740	15
745	12
750	14
755	11
760	6
765	7
770	10
775	13
780	6
785	14
790	10
795	14
800	11
805	9
810	7
815	13
820	12
825	10
830	7
835	12
840	3
845	8
850	8
855	7
860	12
865	7
870	6
875	7
880	6
885	10
890	4
895	15
900	14
905	5
910	3
915	4
920	9
925	4
930	5
935	6
940	4
945	10
950	6
955	7

960	3
965	4
970	7
975	3
980	7
985	7
990	8
995	8
1000	5
1005	1
1010	7
1015	5
1020	3
1025	5
1030	6
1035	6
1040	4
1045	3
1050	4
1055	3
1060	4
1065	1
1070	5
1075	3
1080	2
1085	4
1090	3
1095	5
1100	3
1105	3
1110	3
1115	5
1120	4
1125	5
1130	1
1135	3
1140	5
1145	7
1150	4
1155	6
1160	3
1165	5
1170	5
1175	6
1180	7
1185	8
1190	3
1195	3

1200	4

D Lead Absorption Data

Thickness (cm)	Counts	Average Sheet Thickness (m	Trial 1	Trial 2	Trial 3
0.0000	397	0			
1.7700	361	1.77	1.92	1.89	1.98
5.8700	386	4.1	3.77	4.2	4.81
7.5233	360	1.653333333	1.84	1.81	1.82
10.1367	323	2.613333333	2.78	2.74	2.77
11.8133	270	1.676666667	1.84	1.84	1.83

Minus Ziplock Bag
0.16
0.16
0.17
0.15
0.16

E Inverse Square Law Data

Distance (cm)	Counts
3.00	1217
4.00	872
5.00	629
6.00	479
7.00	412
8.00	344

F Python Code

In [8]:
```
# import libraries
```

Background Radiation

In [187…
```
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#1 Import data
data = pd.read_excel('Radioactive_Decay_template.xlsx',
                                sheet_name = 'Background')
Bdata = data['Latest: Radiation (Counts)'].to_numpy()
#Display data
display(Bdata)
# Exporting table...
data.to_latex("Bdata_table.tex", index=False)
```

```
array([2, 0, 5, 0, 6, 3, 4, 3, 1, 3, 1, 6, 2, 1, 3, 2, 2, 4, 1, 3, 3, 5,
       0, 3, 0, 1, 2, 1, 5, 3, 7, 4, 3, 4, 1, 5, 2, 3, 1, 0, 1, 4, 4, 3,
       3, 1, 3, 2, 3, 4, 3, 1, 2, 2, 2, 3, 4, 1, 2, 4, 7, 1, 4, 2, 2, 3,
       2, 0, 0, 3, 3, 2, 2, 5, 3, 1, 2, 5, 5, 3, 3, 2, 2, 4, 3, 4, 3, 5,
       3, 2, 3, 0, 2, 3, 3, 2, 3, 2, 3, 4])
```

In [176…
```
#2 Convert data from list to array(Not needed...)
#data = np.array(Bdata)
N = len(Bdata)

#3 Compute mean and sample standard deviation of the background counts
bkgAvg = np.mean(Bdata)
bkgStd = np.std(Bdata,ddof=1)

# Uncertainty in the mean
bkgErr = bkgStd / np.sqrt(N)

print(N)
print (bkgAvg)
print (bkgStd)
print (bkgErr)
type(data)
```

```
100
2.68
1.5431831198578285
0.15431831198578286
```

Out[176…
```
pandas.core.frame.DataFrame
```

Create an histogram of the background [also called a frequency plot]

The data you collected represents a vector containing the number of counts detected (n) in each time interval. The best way to visualize this kind of data is in a histogram - a plot of the number of times (f_k) that we measured $n = k$ for each possible integer k.

As an example, assume you made 10 measurements and recorded the following counts: *n = [1 1 0 2 2 3 1 1 2 2]*. In this simple case, we have data that spans the range of integers k = [0,3] (inclusive). So our k vector will be *k = [0 1 2 3]* and our frequencies would be *f_k = [1 4 4 1]*.

Here are some functions that you may find useful:

Syntax: *k = numpy.arange(start, stop, step)*

This function creates a vector of evenly spaced numbers in the range *[start, stop)*, with step size *step*. By default, *step = 1*. Notice that the *stop* limit is excluded from the range, so to create the vector [0, 1, 2, 3], for example, you would use: *k = numpy.arange(0,4)*

The function

Syntax: *counts, bin_edges = numpy.histogram(n, k)*

returns two output arrays from NumPy:

• counts: An array of the number of data points that fall into each bin.

• bin_edges: A float array of length len(counts) + 1 containing the lower and upper boundaries of the bins.

Reference: *https://numpy.org/devdocs/reference/generated/numpy.histogram.html*

The matplotlib *hist* function both plots the data in a histogram, and computes the frequencies associated with each bin:

Syntax: *f_k, _ , _ = ax.hist(n, bins, color='skyblue', edgecolor = 'black')*

The first output *f_k* is the values of the histogram bins. Here, *n* is the input data, and *bins* defines the number and size of histogram bins to generate. If *bins* is a vector, it defines the edges of each histogram bin *including both the leftmost and rightmost edges*. This creates a subtle difference between the *bins* vector and the *k* vector. If you want your histogram bins to be centered on the integer values in *k*, *bins* will need to be offset by -0.5 and one element longer. In other words, if *k = [0 1 2 3]*, you'd want the *bins* vector to be *bins = [-0.5 0.5 1.5 2.5 3.5]*.

Reference: *https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html*

In [158…
```
#4 Define a figure to plot your data and line of best fit later in the file
fig1, ax1 = plt.subplots()

type(Bdata)
```

```
# Create an array of bins centered on integer values that span the entire ra
# Create an array of bins centered on integer values

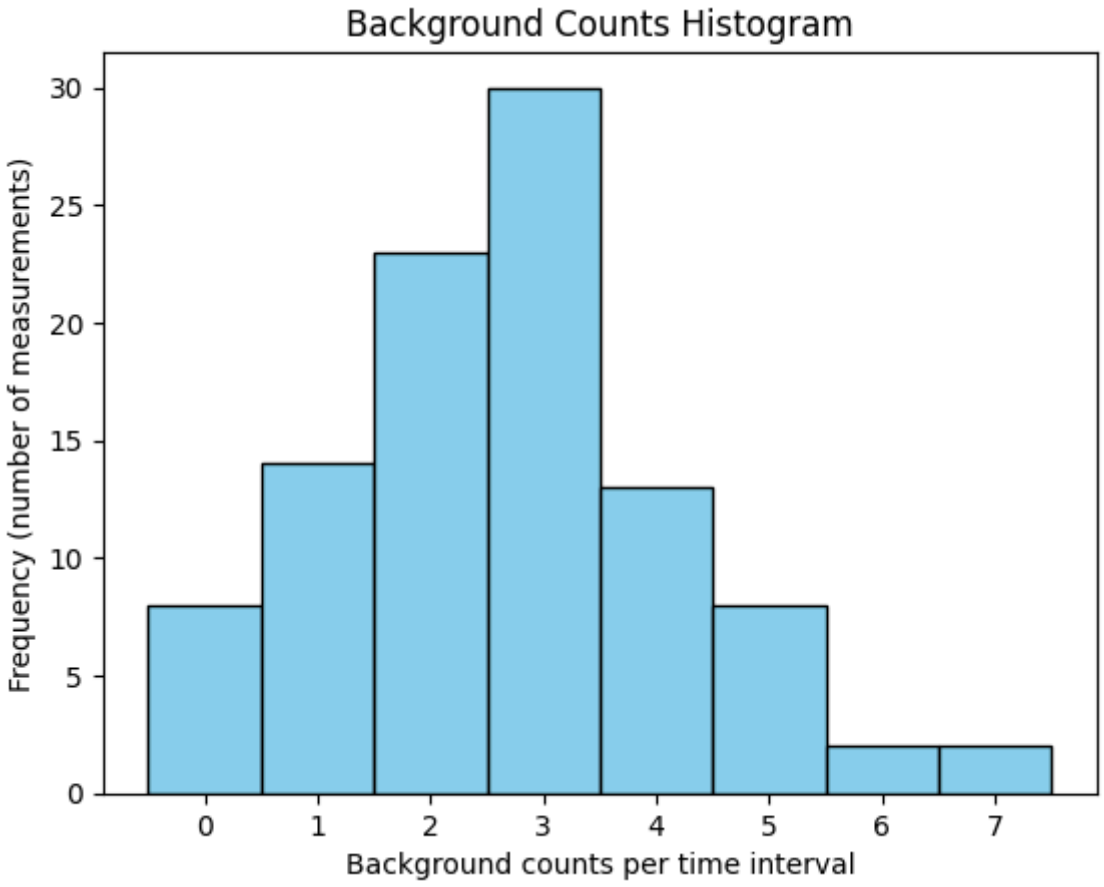
k = np.arange(Bdata.min(),
              Bdata.max() + 1)
bins =  np.arange(Bdata.min() - 0.5,
                  Bdata.max() + 1.5,
                  1)

#5 Create a histogram and plot it
f_k, _, _ = ax1.hist(Bdata, bins=bins, color="skyblue", edgecolor="black")

ax1.set_xlabel("Background counts per time interval")
ax1.set_ylabel("Frequency (number of measurements)")
ax1.set_title("Background Counts Histogram")

fig1.savefig("background_histogram.png", bbox_inches="tight")

plt.show()
```



Compare your histogram to a Poisson distribution

To compare your measurements to a Poisson distribution, overlay the frequency that you would expect to measure from a poisson distribution with the same average counts. The expected frequency is $f_k = Np_k$, where N is the total number of counts, and p_k is the Poisson probability distribution function with mean α given by:

$$p_k = \frac{\alpha^k}{k!} e^{-\alpha}$$

Note that the python package scypi.special has a built-in function to compute factorials:

Syntax: *from scipy.special import factorial*

A poisson distribution has the characteristic that $\sigma^2 = \alpha$ (the variance equals the mean). Is this true for your dataset?

In [175…

```
#6 Plot the expected bin frequencies on the same axes as your histogram
from scipy.special import factorial

bkgAvg = np.mean(Bdata)
N = len(Bdata)

# Calculate expected bin frequencies
expected_frequency = N * bkgAvg**k * np.exp(-bkgAvg) / factorial(k)

# Plot on the same axes as your histogram
ax1.plot(k, expected_frequency, "ro-", label="Poisson prediction")

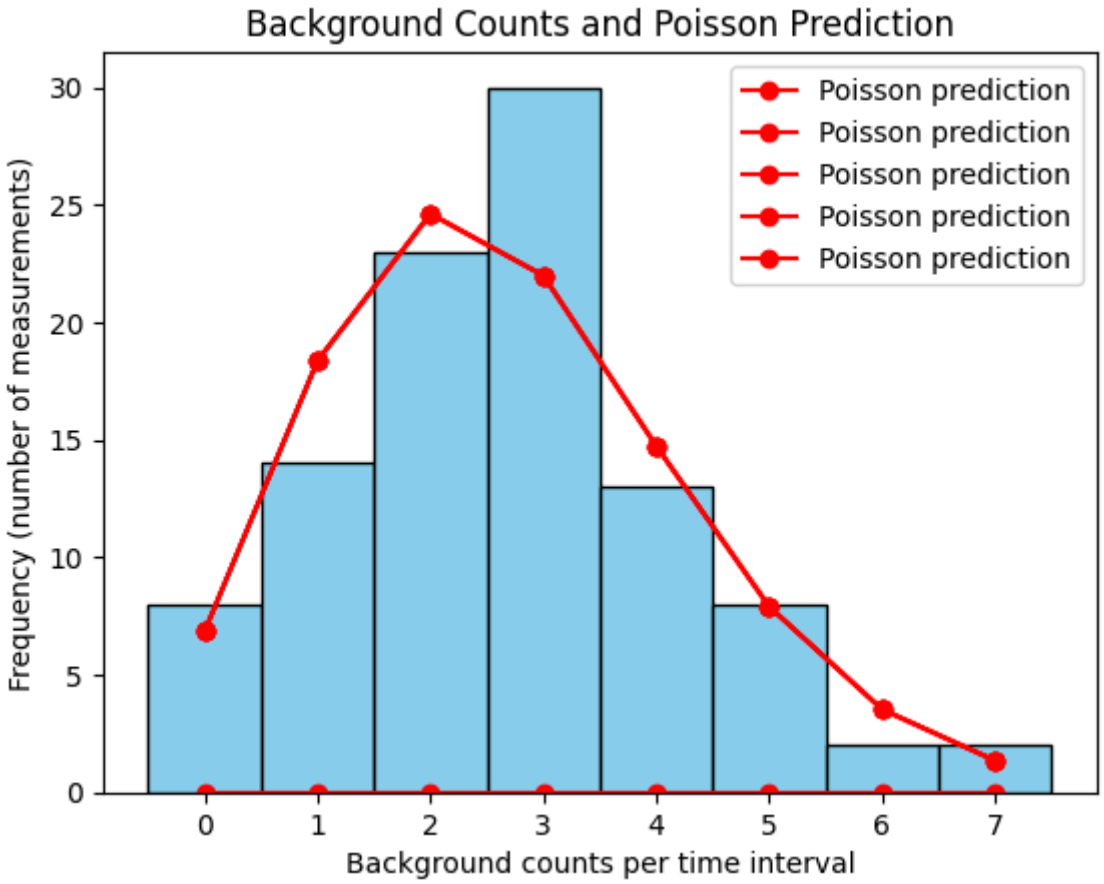
# Label axes and add a title
ax1.set_xlabel("Background counts per time interval")
ax1.set_ylabel("Frequency (number of measurements)")
ax1.set_title("Background Counts and Poisson Prediction")
ax1.legend()

fig1.savefig("BG_histogram with Fit.png", bbox_inches="tight")

display(fig1)

# Mean, Sample Variance, Standard Deviation, and Uncertainty in the Mean
bkgAvg = np.mean(Bdata)
bkgVar = np.var(Bdata, ddof=1)
bkgStd = np.std(Bdata)
# Uncertainty in the mean
bkgErr = bkgStd / np.sqrt(N)

print("Mean:", bkgAvg)
print("Sample Variance:", bkgVar)
print("Standard Deviation:", bkgStd)
print("Uncertainty in the Mean:", bkgStd)
```




```
Mean: 2.68
Sample variance: 2.381414141414141
Standard Deviation: 1.5354478174135386
```

The half life of ^{137m}Ba

In [188…

```
# Continue analyzing the rest of your Radioactive decay data
# 1. Extract half-life time and raw counts

# Load the HALF-LIFE sheet
data = pd.read_excel(
    'Radioactive_Decay_template.xlsx',
    sheet_name='Half-Life Ba137'
)
#Exporting table...
data.to_latex("HL_table.tex", index=False)

time = data['Time (s)'].to_numpy()
HLdata = data['Latest: Radiation (Counts)'].to_numpy()

# 2. Detector settings
tau = 90e-6   # Dead time: 90 microseconds, in seconds
dt = 5       # Each measurement lasts 5 seconds

# 3. Correct the half-life counts
HL_corrected = HLdata / (1 - (HLdata / dt) * tau)
```

```
# 4. Correct the BACKGROUND counts
# Bdata must contain your raw Part A background measurements!
background_corrected = Bdata / (1 - (Bdata / dt) * tau)

# Average corrected background counts per 5 seconds
bkgAvg_corrected = np.mean(background_corrected)

# 5. Subtract the corrected average background
corrected_counts = HL_corrected - bkgAvg_corrected

# 6. Plot
fig, ax = plt.subplots()

ax.plot(time, corrected_counts, 'o', markersize=3, label = "Data")

ax.set_xlabel('Time (s)')
ax.set_ylabel('Background-subtracted counts per 5 s')
ax.set_title('Barium-137m Decay — Dead-Time Corrected')

#plt.show()

# using curve_fit to fit the data
from scipy.optimize import curve_fit
def halfLife(t,A,L):
    return A*np.e**(-L*t)

popt, pcov = curve_fit(halfLife, time, corrected_counts, [250, 0.003466])
#print(popt)
A = popt[0]
L = popt[1]
print(A)
print(L)
def HLfit(t):
    return A*np.e**(-L*t)

t = np.linspace(0, 1200, 1200)
y = HLfit(t)

ax.plot(t, y, label='Exponential Fit')
ax.legend()
fig.savefig("Half-Life of Ba.png", dpi=300, bbox_inches="tight")

plt.show()

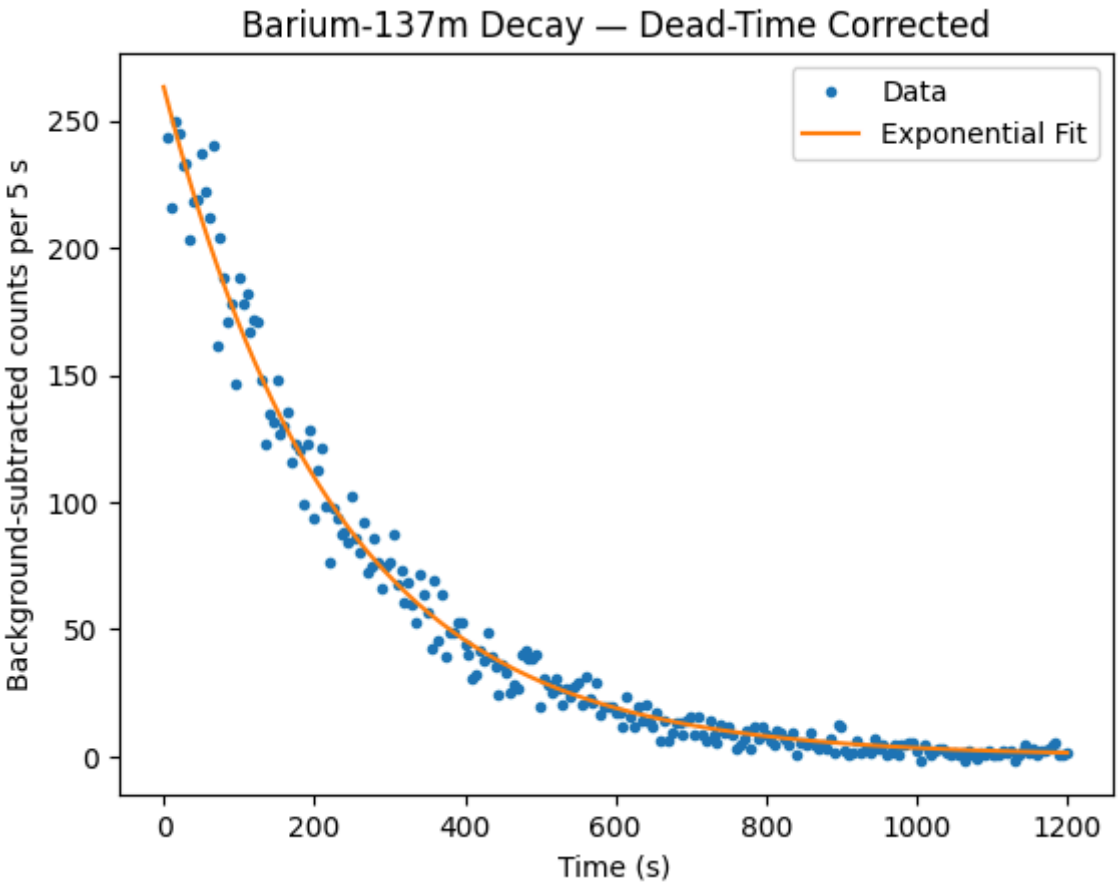
#Error on parameters...
HLpcov = np.sqrt(np.diag(pcov))

# Mean, Sample Variance, Standard Deviation, and Uncertainty in the Mean
HLAvg = np.mean(HLdata)
HLVar = np.var(HLdata, ddof=1)
HLStd = np.std(HLdata)
# Uncertainty in the mean
NHL = len(HLdata)
print(NHL)
HLErr = HLStd / np.sqrt(NHL)
```

```
print("Mean:", HLAvg)
print("Sample Variance:", HLVar)
print("Standard Deviation:", HLStd)
print("Uncertainty in the Mean:", HLErr)
print("Error on each Parameters:", HLpcov)


#Lambda is 0.0043773832515909585 so Half Life is approximately
#HL = ln2/0.0043773832515909585 is approximately 158.35 seconds
```

```
263.2813650787458
0.0043773832515909585
```




```
240
Mean: 51.50416666666667
Sample Variance: 4111.999982566248
Standard Deviation: 63.9911450851253
Uncertainty in the Mean: 4.1306106536572145
Error on each Parameters: [2.47516096e+00 5.76137806e-05]
```

Pb Absorption coefficient

In [189…

```
#Loading...
AC = pd.read_excel(
    'Radioactive_Decay_template.xlsx',
    sheet_name='Pb Absorption Coeff'
)

# Exporting table...
```

```
AC.to_latex("AC_table.tex", index=False)

display(AC)
thickness = AC['Thickness (cm)'].to_numpy()
Counts = AC['Counts'].to_numpy() - bkgAvg_corrected
#BGCorrected = Counts - Bdata
#print(thickness)
#print(Counts)
#plt.yticks(np.arange(0, 401, 10))

fig, ax = plt.subplots()

ax.plot(thickness, Counts, 'o', markersize=10, label='Data')


ax.set_xlabel('Time (s)')
ax.set_ylabel('Counts')
ax.set_title('Lead Absorbtion - Dead-Time Corrected')

#plt.plot(thickness, Counts, 'o', markersize=10)
#plt.show()

#fitting...
#I_naught...
print
I_o = 397 - bkgAvg_corrected
def I(x, u):
    return I_o*np.exp(-u*x)

popt, pcov = curve_fit(I, thickness, Counts, [1])

print(pcov)
U = popt[0]
print("The Absorbtion Coefficent is:", U)
def Ifit(t):
    return I_o*np.exp(-U*t)

t = np.linspace(0, 1200, 1200)
y = Ifit(t)

ax.plot(t, y, label='Exponential Fit')
fig.savefig("Lead Absorbtion with Fit.png", dpi=300, bbox_inches="tight")

plt.show()

#Errors...

#Error on parameters...
Pbpcov = np.sqrt(np.diag(pcov))

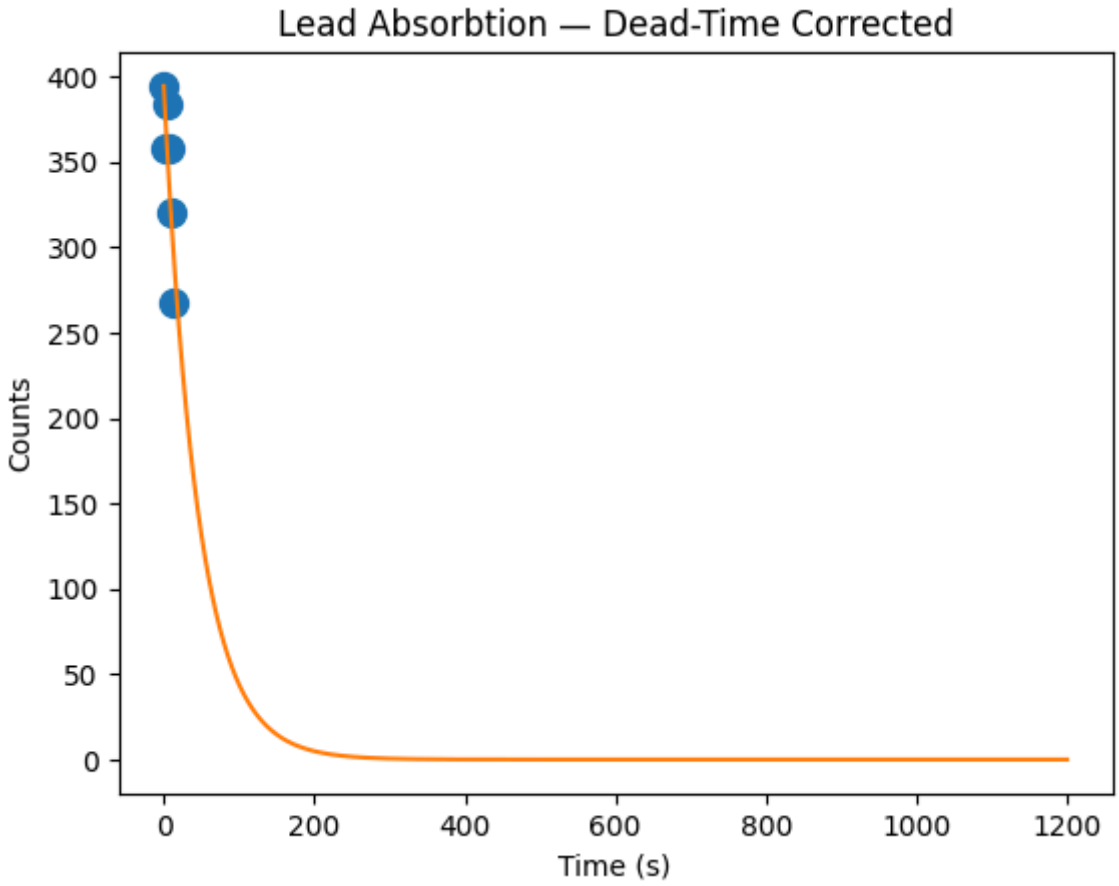
# Mean, Sample Variance, Standard Deviation, and Uncertainty in the Mean
#PbAvg = np.mean(HLdata)
#PbVar = np.var(HLdata, ddof=1)
#PbStd = np.std(HLdata)
# Uncertainty in the mean
#NHL = len(HLdata)
```

```
#print(NHL)
#HLErr = HLStd / np.sqrt(NHL)

#print("Mean:", HLAvg)
#print("Sample Variance:", HLVar)
#print("Standard Deviation:", HLStd)
#print("Uncertainty in the Mean:", HLErr)
print("Error on each Parameters:", Pbpcov)
```

	Thickness (cm)	Counts	Average Sheet Thickness (mm)	Trial 1	Trial 2	Trial 3	Minus Ziplock Bag
0	0.000000	397	0.000000	NaN	NaN	NaN	NaN
1	1.770000	361	1.770000	1.92	1.89	1.98	0.16
2	5.870000	386	4.100000	3.77	4.20	4.81	0.16
3	7.523333	360	1.653333	1.84	1.81	1.82	0.17
4	10.136667	323	2.613333	2.78	2.74	2.77	0.15
5	11.813333	270	1.676667	1.84	1.84	1.83	0.16

```
[[2.17131384e-05]]
The Absorbtion Coefficent is: 0.021932414677133864
```




```
Error on each Parameters: [0.00465974]
```

Inverse-square law

In [190…

```
#Load Data...
ISL = pd.read_excel(
    'Radioactive_Decay_template.xlsx',
    sheet_name='Inverse Square Law'
)
display(ISL)
# Exporting table...
ISL.to_latex("ISL_table.tex", index=False)

Dist = ISL['Distance (cm)'].to_numpy()
#the Background Average is subtracted...
Counts = ISL['Counts'].to_numpy() - bkgAvg_corrected*40 #per 5s times 40 = 2

#the geiger counter's dead time is ommitted because it is neglible

fig, ax = plt.subplots()

ax.plot(Dist, Counts, 'o', markersize=10)


ax.set_xlabel('Distance (cm)')
ax.set_ylabel('Counts')
ax.set_title('Inverse Square Law – Background Corrected')

#fitting...
#ISL...
#guessing for a using rearanged equation...
#y=a/(x-b)**n + c => a = y*x**2
a_guess = Counts[0] * Dist[0]**2
#print(a_guess)

#def ISL(x,a,b,c,n):
#    return a/(x-b)**n + c

#popt, pcov = curve_fit(ISL, Dist, Counts, [a_guess,0,0,2], maxfev=20000)
#print(Dist)
#print(Counts)

### Given equation not working!!! Manually setting b and c to zero...

def ISL2(x,a,n):
    return a/(x)**n

popt, pcov = curve_fit(ISL2, Dist, Counts, [a_guess,2], maxfev=20000)

#print(popt)
A = popt[0]
N = popt[1]

print("A is ",A)
print("N is ",N)

def ISL3(x):
    return A/(x)**N
```

```
t = np.linspace(Dist.min(), Dist.max(), 200)
y = ISL3(t)

ax.plot(t, y)
fig.savefig("Inverse Square Law with Fit.png", dpi=300, bbox_inches="tight")
plt.show()

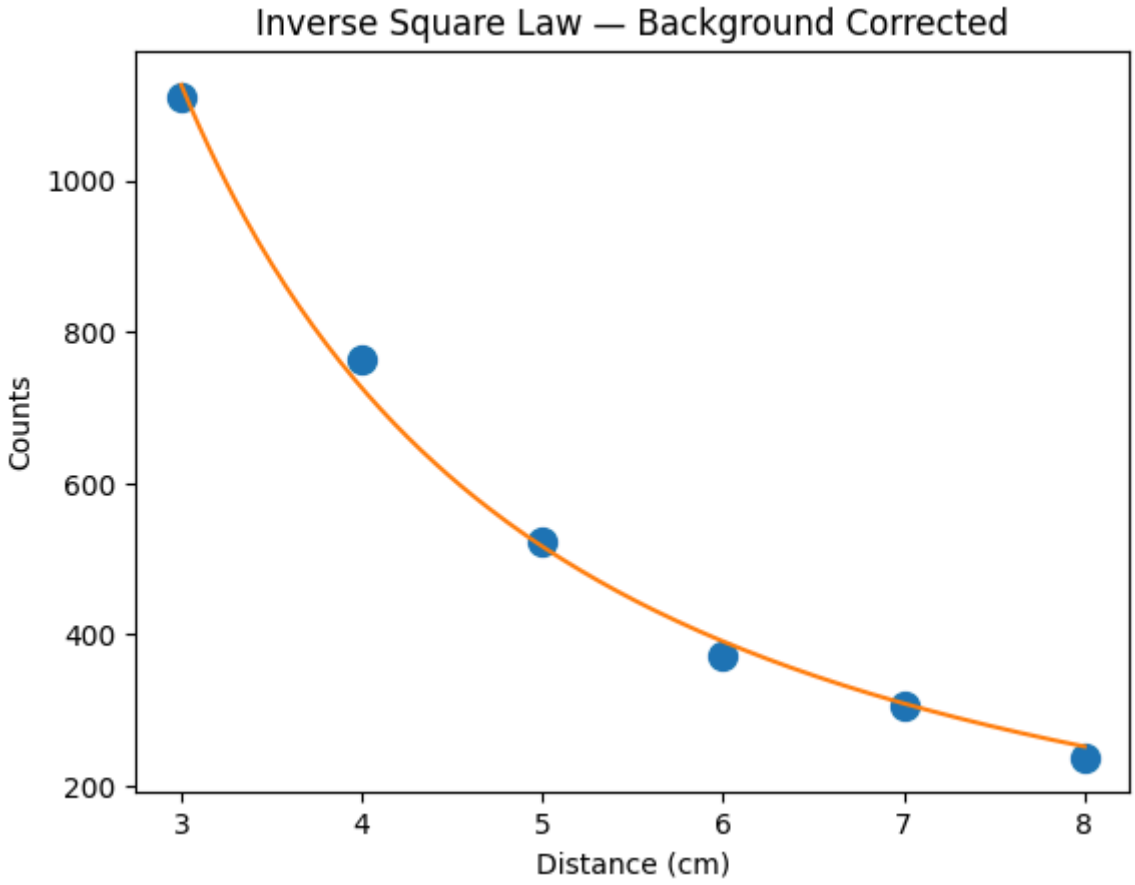
print("Finally, n is calculated to be:", N)

#Errors...

ISLpcov = np.sqrt(np.diag(pcov))
print("Error on each Parameters:", ISLpcov)
```

	Distance (cm)	Counts
0	3	1217
1	4	872
2	5	629
3	6	479
4	7	412
5	8	344

```
A is  6043.437008218625
N is  1.5288540467446783
```



```
Finally, n is calculated to be: 1.5288540467446783
Error on each Parameters: [4.56853766e+02 5.57902192e-02]
```

In [ ]: